

ARTIFICIAL INTELLIGENCE

Natural Language Processing LLM Module

Lecture Notes

Master's Program in Artificial Intelligence

ACADEMIC YEAR

2025–2026

JACOPO PARRETTI

Contents

I	Foundations of Neural Networks	3
1	Introduction to Neural Networks	3
1.1	The Concept of a Neuron	3
1.2	Activation Functions	3
1.3	Multi-Layer Neural Networks	4
2	Learning in Neural Networks	5
2.1	The Learning Problem	5
2.2	Optimization Methods	5
2.3	From Numbers to Text	5
3	Introduction to Transformers	6
3.1	Why Transformers?	6
3.2	Transfer Learning and Foundation Models	7
3.3	Self-Supervised Learning	7
3.4	The Scaling Hypothesis	7
4	The Attention Mechanism	8
4.1	Motivation: Context-Dependent Representations	8
4.2	Attention vs. Standard Neural Networks	8
5	Transformer Processing: Mathematical Framework	9
5.1	Notation and Terminology	9
5.2	Transformer Layer: The Main Building Block	10
6	Attention Coefficients	11
6.1	The Goal of Attention	11
6.2	Linear Combination Case	11
7	Self-Attention Mechanism	11
7.1	How to Compute Attention Coefficients	11
7.2	Self-Attention in Transformers	12
7.3	Summary: Self-Attention Transformation	12
8	Learnable Parameters in Self-Attention	13
8.1	The Problem with Simple Self-Attention	13
8.2	Solution: Query, Key, and Value Transformations	13
8.3	Modified Self-Attention Formula	14
8.4	General Formula and Dimensions	14
8.5	Key Distinction: Data-Dependent Attention	15
9	Multi-Head Attention	16
9.1	Limitation of Single Attention Head	16
9.2	The Multi-Head Solution	16
10	Positional Encoding	17
10.1	The Need for Position Information	17
10.2	Solution: Positional Encoding	17
10.3	Properties of Good Positional Encoding	18

11 The Complete Transformer Architecture	18
11.1 Encoder-Decoder Structure	18
11.2 Layer Normalization and Residual Connections	18
12 Transformer Layers	19
12.1 Multi-Head Attention Algorithm	19
12.2 Transformer Layer Structure	19
12.3 Nonlinearity and Feed-Forward Network	20
12.4 Complete Transformer Layer	20
12.5 Transformer Layer Algorithm	21
13 Computational Complexity of the Attention Layer	22
13.1 Attention Layer Complexity	22
13.2 Comparison with Fully Connected Network	22
14 Tokenization: One-Hot Representation	22
14.1 How to Compute Token Vectors for a Given Text?	22
14.2 One-Hot Representation	22
15 Tokenization: Byte Pair Encoding	23
15.1 Motivation	23
15.2 Byte Pair Encoding (BPE)	23
15.3 Dictionary Generation with Byte Pair Encoding	23
15.4 Text Tokenization	24
16 Transformer Architectures	24
16.1 Introduction to Transformer Architectures	24
16.2 Decoder Transformers	25

Part I

Foundations of Neural Networks

1 Introduction to Neural Networks

1.1 The Concept of a Neuron

Neural networks are computational models inspired by biological neural systems. At their core, they consist of interconnected units called **neurons** (or **nodes**).

Definition: Artificial Neuron

An **artificial neuron** is a mathematical function that:

1. Receives multiple inputs $(x_1, x_2, \dots, x_n)$
2. Computes a weighted sum of these inputs plus a bias term
3. Applies an activation function to produce an output

1.1.1 Mathematical Formulation

The computation performed by a single neuron can be expressed as:

$$z = \sum_{i=1}^n w_i x_i + b = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

where:

- x_i are the input values
- w_i are the **weights** associated with each input
- b is the **bias** term
- z is the weighted sum (also called the **pre-activation**)

The final output of the neuron is obtained by applying an **activation function** σ to the weighted sum:

$$y = \sigma(z) = \sigma\left(\sum_{i=1}^n w_i x_i + b\right)$$

1.2 Activation Functions

Activation functions introduce non-linearity into the network, enabling it to learn complex patterns. Common activation functions include:

Common Activation Functions

1. Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Outputs values in the range $(0, 1)$. Often used for binary classification.

2. ReLU (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0, z)$$

Outputs z if $z > 0$, otherwise 0. Most commonly used in modern deep learning.

3. Tanh (Hyperbolic Tangent):

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Outputs values in the range $(-1, 1)$. Centers data around zero.

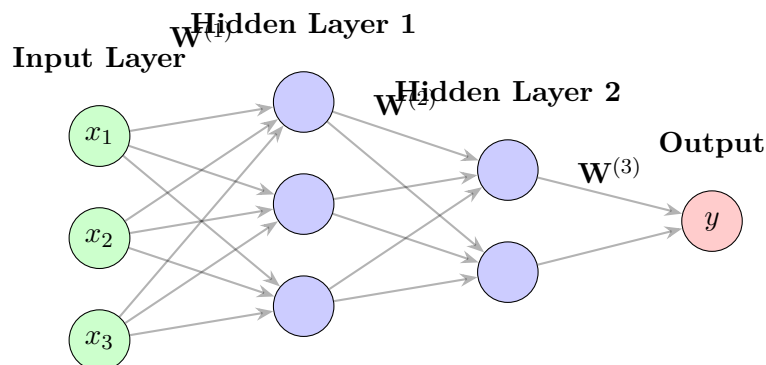
1.3 Multi-Layer Neural Networks

The power of neural networks comes from connecting multiple neurons in **layers** to form deep architectures.

Definition 1.1 (Feedforward Neural Network). A **feedforward neural network** (also called a **multilayer perceptron**) consists of:

- An **input layer**: receives the input features
- One or more **hidden layers**: perform intermediate computations
- An **output layer**: produces the final prediction

1.3.1 Architecture Visualization



1.3.2 Weight Matrices

Each layer in a neural network is associated with a **weight matrix** that defines the connections between neurons.

- $\mathbf{W}^{(1)}$: Weight matrix connecting the input layer to the first hidden layer
- $\mathbf{W}^{(2)}$: Weight matrix connecting the first hidden layer to the second hidden layer
- $\mathbf{W}^{(3)}$: Weight matrix connecting the second hidden layer to the output layer

For a layer with n input neurons and m output neurons, the weight matrix $\mathbf{W}$ has dimensions $m \times n$.

2 Learning in Neural Networks

2.1 The Learning Problem

Objective: Learning the Weights

The fundamental goal of training a neural network is to **learn the optimal values of the weight matrices** $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \dots, \mathbf{W}^{(L)}$ that minimize the prediction error on the training data.

This is analogous to learning the coefficients in a regression model:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

where $\beta_0, \beta_1, \dots, \beta_n$ are the parameters to be learned.

2.2 Optimization Methods

Several optimization algorithms are used to learn neural network weights:

2.2.1 Gradient Descent

Definition 2.1 (Gradient Descent). **Gradient descent** is an iterative optimization algorithm that updates the weights in the direction opposite to the gradient of the loss function:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}$$

where:

- η is the **learning rate** (step size)
- $\nabla_{\mathbf{W}} \mathcal{L}$ is the gradient of the loss function with respect to the weights

2.2.2 ADAM Optimizer

ADAM (Adaptive Moment Estimation)

ADAM is an advanced optimization algorithm that combines ideas from momentum and adaptive learning rates. It is one of the most popular optimizers in modern deep learning.

Key features:

- Adapts learning rates for each parameter individually
- Uses moving averages of gradients (momentum)
- Computationally efficient and requires little memory
- Works well in practice with default hyperparameters

2.3 From Numbers to Text

Observation. While neural networks fundamentally operate on **numerical data**, they can be adapted to work with various data types, including:

- **Images:** Represented as matrices of pixel values
- **Audio:** Represented as waveforms or spectrograms
- **Text:** Represented through embeddings (numerical vectors)

This versatility enables neural networks to perform tasks such as **text generation**, **machine translation**, and **image captioning**.

3 Introduction to Transformers

Key Concept: Transformers

All of the preceding discussion serves as foundation for understanding a specific and revolutionary type of neural network architecture: **Transformers**.

Transformers are the backbone of modern large language models (LLMs) such as GPT, BERT, and many others. They have revolutionized natural language processing and are now being applied to various domains beyond text.

3.1 Why Transformers?

Transformers represent one of the most important developments in deep learning. First introduced in the field of natural language processing, they have greatly surpassed previous architectures such as **Recurrent Neural Networks (RNNs)** in both performance and efficiency.

3.1.1 Core Concept: Attention Mechanism

The Attention Mechanism

Transformers are based on a processing concept called **attention**. The attention mechanism allows the model to focus on different parts of the input when producing each part of the output, weighing the importance of different input elements dynamically.

Why "Transformers"?

The name comes from their ability to **transform input vectors** from one representation space to another, creating richer internal representations suitable for downstream tasks.

3.1.2 Input Flexibility

Transformers can process various types of inputs:

- **Unstructured sets of vectors:** No inherent ordering required
- **Ordered sequences:** Natural for text and time series
- **General representations:** Broad applicability across domains

3.2 Transfer Learning and Foundation Models

Transfer Learning with Transformers

Transfer learning is particularly effective with Transformers. The typical workflow involves:

1. **Pre-training:** Train a large model on a massive corpus of data
2. **Fine-tuning:** Adapt the pre-trained model to specific downstream tasks with smaller, task-specific datasets

This approach has led to the concept of **foundation models**—large-scale models trained on broad data that serve as a foundation for various applications.

3.3 Self-Supervised Learning

Observation. Transformers can be trained in a **self-supervised** manner using unlabeled data. This is crucial because:

- Labeled data is expensive and time-consuming to obtain
- Unlabeled text data is abundant (e.g., web pages, books, articles)
- Self-supervised objectives (e.g., predicting masked words) provide strong learning signals

3.4 The Scaling Hypothesis

Scaling Hypothesis

The **scaling hypothesis** states that by increasing both:

- The **scale of the model** (number of learnable parameters)
- The **size of the training dataset**

significant improvements in model performance can be achieved.

This hypothesis has been empirically validated across numerous experiments, leading to increasingly large models.

3.4.1 Massive Scale and Parallel Processing

Transformers are particularly well-suited to **massively parallel processing**, which enables:

- Training of very large models with parameters on the order of **trillions** (10^{12})
- Reasonable training times despite model size (using distributed computing and specialized hardware like GPUs and TPUs)
- Efficient inference for real-world applications

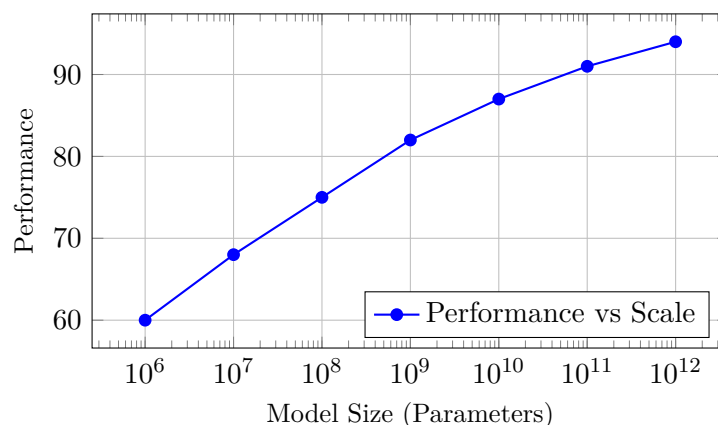


Illustration: As model size increases (log scale), performance improves following the scaling hypothesis.

4 The Attention Mechanism

4.1 Motivation: Context-Dependent Representations

Example 4.1 (Word Ambiguity). Consider the following two sentences containing the word "bank":

- *I swam across the river to get to the other **bank***
- *I walked across the road to get cash from the **bank***

To determine the exact interpretation of the word *bank*, an artificial neural network should **rely more heavily on** (i.e., **attend to**) specific words from the rest of the sentence (**context**).

In the first sentence, "bank" should attend more to words like "river" and "swam" (indicating a riverbank). In the second sentence, "bank" should attend more to words like "cash" and "road" (indicating a financial institution).

4.2 Attention vs. Standard Neural Networks

Key Difference

Standard neural networks: When the network is trained, weights are **fixed**.

Attention mechanism: Uses **weighting factors** whose values **depend on the specific input data**.

Observation. A Transformer can be seen as a **rich form of embedding** in which a given vector (e.g., representing a word) is mapped to a location that **depends on the other vectors** (i.e., words) of the sentence.

Example: The embedding of the word "bank" will be:

- Close to "water" in the first sentence
- Close to "money" in the second sentence

This context-dependent representation is what makes Transformers so powerful for natural language understanding.

5 Transformer Processing: Mathematical Framework

5.1 Notation and Terminology

Definition 5.1 (Input Representation). The input to a Transformer consists of:

- **Set of vectors:** $\{x_n\}$ of dimensionality D where $n = 1, \dots, N$
- Each vector is called a **token**

5.1.1 What is a Token?

A **token** is a fundamental unit of input data. Examples include:

- A **word** in a sentence
- A **patch** in an image
- An **amino acid** in a protein sequence
- A **number or set of numbers** in a time series

5.1.2 Features

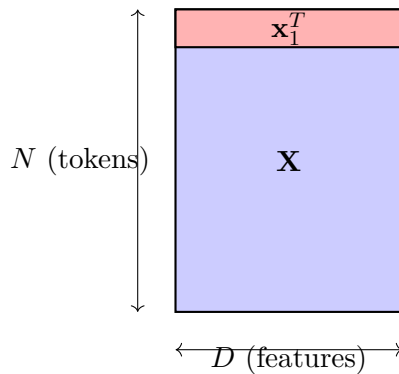
The elements x_{ni} of a token are called **features**, where $i = 1, \dots, D$.

5.1.3 Data Matrix

Definition 5.2 (Data Matrix). The **data matrix** (set of tokens) is denoted by $\mathbf{X}$ with dimension $N \times D$:

- Each **row** is a token vector $\mathbf{x}_n^T$
- N is the number of tokens
- D is the number of features (dimensionality)

A **training set** contains many sets of tokens (i.e., many matrices).



5.2 Transformer Layer: The Main Building Block

Transformer Layer

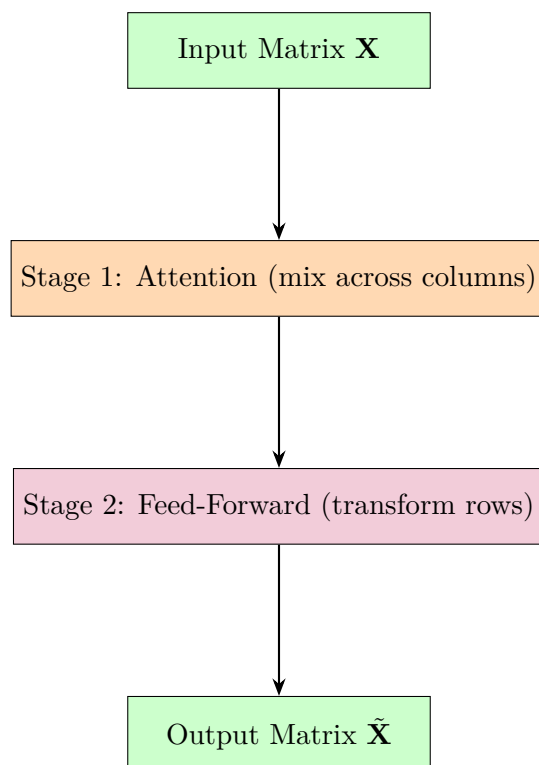
A **Transformer layer** is the fundamental building block of a Transformer that:

- Takes a **data matrix $\mathbf{X}$** as input
- Returns a **transformed matrix $\tilde{\mathbf{X}}$** of the same dimension as output

$$\tilde{\mathbf{X}} = \text{TransformerLayer}[\mathbf{X}]$$

5.2.1 Key Properties

1. **Multiple layers:** Multiple Transformer layers can be applied in succession to learn **rich internal representations**
2. **Learnable weights:** Each Transformer layer contains weights that can be learned by **gradient descent** using an appropriate **cost function**
3. **Two-stage processing:** Each Transformer layer performs two stages:
 - (a) **Stage 1 (Attention):** Mixes features from different tokens across the **columns** of $\mathbf{X}$ (attention mechanism)
 - (b) **Stage 2 (Feed-forward):** Transforms the values of each token acting on the **rows** of $\mathbf{X}$ independently



6 Attention Coefficients

6.1 The Goal of Attention

Attention Objective

Given a set of tokens in an embedding space: $\mathbf{x}_1, \dots, \mathbf{x}_N$

Goal: Map them into another set $\mathbf{y}_1, \dots, \mathbf{y}_N$ with richer semantic structure.

Key principle: The value of a generic output token $\mathbf{y}_n$ should **depend on all input tokens**.

Attention: The dependence should be **stronger** for inputs $\mathbf{x}_m$ particularly important for determining the new representation of $\mathbf{y}_n$.

6.2 Linear Combination Case

In the simplest case, we can express the output as a **linear combination** of the input tokens:

$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m$$

where:

- a_{nm} are called **attention weights**
- Constraint: $a_{nm} \geq 0$ and $\sum_{m=1}^N a_{nm} = 1$ (partition of unity to avoid compensation)

Definition 6.1 (Attention Weights). The coefficients a_{nm} represent how much the n -th output token should "attend to" the m -th input token. They form a probability distribution over the input tokens for each output position.

Interpretation:

- If a_{nm} is large, token $\mathbf{x}_m$ has a strong influence on $\mathbf{y}_n$
- If a_{nm} is small, token $\mathbf{x}_m$ has little influence on $\mathbf{y}_n$
- The sum constraint ensures that the total attention is normalized

7 Self-Attention Mechanism

7.1 How to Compute Attention Coefficients

7.1.1 Example: Movie Selection (Hard Attention)

To understand how attention coefficients work, consider a simplified example of movie selection based on user preferences.

Example 7.1 (Movie Recommendation with Hard Attention). Given:

- **Preference (query):** A user's preference vector representing their interests
- **Movies database:** A collection of movies, each with:
 - **Keys:** Feature vectors describing movie properties
 - **Values:** The actual movie content/representation

The system computes a **similarity score** $Q^T K$ between the user's query and each movie's key. In **hard attention**, only the movie with the highest similarity score is selected.

7.2 Self-Attention in Transformers

In Transformers, the attention mechanism is more sophisticated than the hard attention example above. Here's how it works:

Self-Attention: Key Concepts

- Each input token $\mathbf{x}_n$ serves **both** as a **key** and a **value** (i.e., the token itself summarizes its features)
- The token $\mathbf{x}_m$ is used as a **query** for the output $\mathbf{y}_m$ (then $\mathbf{x}_m$ is compared to all the key tokens)
- The dot product $\mathbf{x}_n^T \mathbf{x}_m$ is used to compute the **query-key similarity**, which determines how much the token represented by $\mathbf{x}_n$ should **attend to** the token represented by $\mathbf{x}_m$ (i.e., how $\mathbf{x}_n$ is important to generate $\mathbf{y}_m$)

7.2.1 Computing Attention Weights with Softmax

Constraints are imposed using the **softmax function**:

$$a_{nm} = \frac{\exp(\mathbf{x}_n^T \mathbf{x}_m)}{\sum_{m'=1}^N \exp(\mathbf{x}_n^T \mathbf{x}_{m'})}$$

This formula ensures:

- All attention weights are positive: $a_{nm} > 0$
- They sum to one: $\sum_{m=1}^N a_{nm} = 1$
- Higher dot products yield higher attention weights (exponential scaling)

7.3 Summary: Self-Attention Transformation

Self-Attention Formula

Each **input vector** $\mathbf{x}_n$ is transformed to a corresponding **output vector** $\mathbf{y}_n$ by taking a **linear combination** of input vectors:

$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m$$

where the weight a_{nm} applied to vector $\mathbf{x}_m$ is computed by the softmax of the dot-product between the **query** $\mathbf{x}_n$ and the **key** $\mathbf{x}_m$.

7.3.1 Matrix Notation

In matrix form, the self-attention operation can be written as:

$$\mathbf{Y} = \text{Softmax}[\mathbf{X}\mathbf{X}^T]\mathbf{X}$$

where:

- $\mathbf{X}$ is the $N \times D$ input matrix (N tokens, D features)
- $\mathbf{X}^T$ is the $D \times N$ transpose
- $\mathbf{X}\mathbf{X}^T$ is an $N \times N$ similarity matrix

- Softmax is applied row-wise to normalize each row
- $\mathbf{Y}$ is the $N \times D$ output matrix

Interpretation of $\mathbf{XX}^T$:

- This is an $N \times N$ matrix where element (n, m) contains $\mathbf{x}_n^T \mathbf{x}_m$
- Each row represents how similar token n is to all other tokens
- After softmax, each row becomes the attention weights for that token

8 Learnable Parameters in Self-Attention

8.1 The Problem with Simple Self-Attention

Observation. The basic self-attention transformation $\mathbf{Y} = \text{Softmax}[\mathbf{XX}^T]\mathbf{X}$ has **no learnable parameters**.

Additionally, each feature value within a token vector $\mathbf{x}_n$ plays an **equal role** in determining the attention coefficients. This limits the expressiveness of the model.

8.2 Solution: Query, Key, and Value Transformations

Modified Feature Vectors

Define **modified feature vectors** as linear transformations of the original vectors:

$$\begin{aligned}\mathbf{Q} &= \mathbf{XW}^{(q)} && \text{(Queries)} \\ \mathbf{K} &= \mathbf{XW}^{(k)} && \text{(Keys)} \\ \mathbf{V} &= \mathbf{XW}^{(v)} && \text{(Values)}\end{aligned}$$

where $\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, and $\mathbf{W}^{(v)}$ are **learnable parameter matrices**.

8.2.1 Dimensions

The weight matrices have the following dimensions:

- $\mathbf{W}^{(q)}$: $D \times D_q$ (not necessarily $D_q = D$)
- $\mathbf{W}^{(k)}$: $D \times D_k$
- $\mathbf{W}^{(v)}$: $D \times D_v$

Usually, $D_k = D_v = D$, so:

- Dimension of output = dimension of input
- **Multiple Transformer layers can be stacked** sequentially

8.2.2 Resulting Matrices

After transformation:

- $\mathbf{Q}$: $N \times D_q$ matrix of query vectors
- $\mathbf{K}$: $N \times D_k$ matrix of key vectors

- $\mathbf{V}$: $N \times D_v$ matrix of value vectors

8.3 Modified Self-Attention Formula

With the learnable transformations, the self-attention mechanism becomes:

$$\mathbf{Y} = \text{Softmax} [\mathbf{Q}\mathbf{K}^T] \mathbf{V}$$

or equivalently:

$$\mathbf{Y} = \text{Softmax} \left[\mathbf{X}\mathbf{W}^{(q)} \left(\mathbf{X}\mathbf{W}^{(k)} \right)^T \right] \mathbf{X}\mathbf{W}^{(v)}$$

Key benefits:

- The model can **learn** which features are important for queries, keys, and values
- Different features can play different roles in computing attention
- The weight matrices $\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, $\mathbf{W}^{(v)}$ are trained via gradient descent

8.4 General Formula and Dimensions

General Self-Attention Formula

The general formula for computing the self-attention transformation is:

$$\mathbf{Y} = \text{Softmax}[\mathbf{Q}\mathbf{K}^T]\mathbf{V}$$

where:

- $\mathbf{Q}\mathbf{K}^T$ has dimension $N \times N$ (attention weight matrix)
- $\mathbf{Y}$ has dimension $N \times D_v$ (output matrix)

8.4.1 Dimensional Analysis

Let's trace through the dimensions step by step:

1. **Input:** $\mathbf{X}$ has dimension $N \times D$

2. **Query transformation:**

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}$$

- $\mathbf{X}$: $N \times D$
- $\mathbf{W}^{(q)}$: $D \times D$
- $\mathbf{Q}$: $N \times D$

3. **Key transformation:**

$$\mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}$$

- $\mathbf{X}$: $N \times D$
- $\mathbf{W}^{(k)}$: $D \times D$
- $\mathbf{K}$: $N \times D$

4. **Attention scores:**

$$\mathbf{Q}\mathbf{K}^T$$

- $\mathbf{Q}$: $N \times D$
- $\mathbf{K}^T$: $D \times N$
- $\mathbf{QK}^T$: $N \times N$

5. Value transformation:

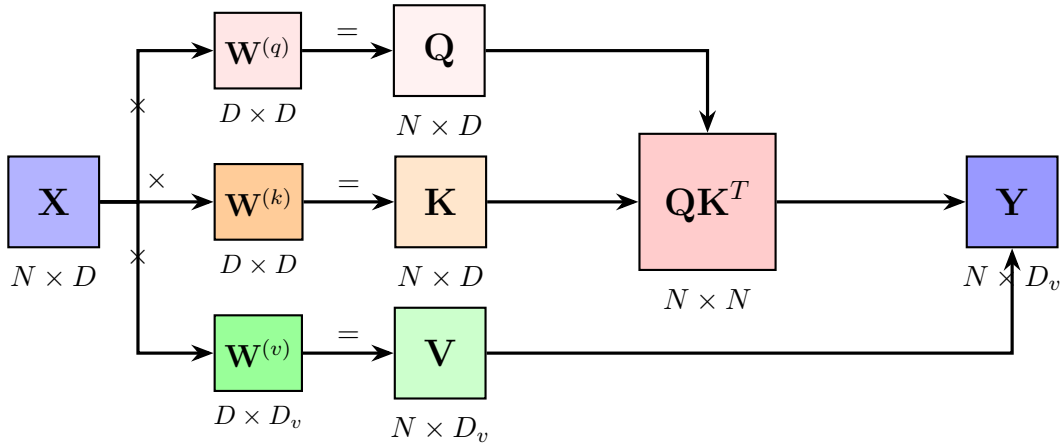
$$\mathbf{V} = \mathbf{XW}^{(v)}$$

- $\mathbf{X}$: $N \times D$
- $\mathbf{W}^{(v)}$: $D \times D_v$
- $\mathbf{V}$: $N \times D_v$

6. Final output:

$$\mathbf{Y} = \text{Softmax}[\mathbf{QK}^T]\mathbf{V}$$

- $\text{Softmax}[\mathbf{QK}^T]$: $N \times N$
- $\mathbf{V}$: $N \times D_v$
- $\mathbf{Y}$: $N \times D_v$



8.5 Key Distinction: Data-Dependent Attention

Important Notice

Standard neural networks multiply activations by **fixed weights**.
Transformers multiply activations by **data-dependent attention coefficients**.
 This is a fundamental difference that gives Transformers their power and flexibility.

8.5.1 Example: Input-Specific Attention

Example 8.1 (Selective Attention). Consider two different input vectors $\mathbf{x}'$ and $\mathbf{x}''$:

- An attention coefficient could be **close to zero** for input vector $\mathbf{x}'$
 - This means the incoming signal is **ignored** in the signal path
- The same attention coefficient could be **very large** for another input vector $\mathbf{x}''$
 - This means the incoming signal is **strongly considered**

Key insight: By contrast, if a standard neural network learns to ignore an input, it does so **for all input vectors**. The network cannot selectively attend based on context.

Observation. This data-dependent weighting allows Transformers to:

- **Dynamically route information** based on input content
- **Context-sensitive processing:** The same token can be processed differently depending on surrounding tokens
- **Flexible feature selection:** Different features become important for different inputs
- **Adaptive computation:** The network's effective structure changes with each input

This is why Transformers excel at tasks requiring understanding of context and relationships between elements, such as natural language processing and sequence modeling.

9 Multi-Head Attention

9.1 Limitation of Single Attention Head

Observation. While self-attention provides a powerful mechanism for capturing relationships between tokens, a single attention head may not capture all types of relationships effectively. Different aspects of the input may require different attention patterns.

9.2 The Multi-Head Solution

Multi-Head Attention

Multi-head attention allows the model to attend to information from different representation subspaces simultaneously. Instead of computing a single attention function, we compute h parallel attention functions (heads).

9.2.1 Mathematical Formulation

For each head $i = 1, \dots, h$:

$$\begin{aligned}\mathbf{Q}^{(i)} &= \mathbf{XW}^{(q,i)} \\ \mathbf{K}^{(i)} &= \mathbf{XW}^{(k,i)} \\ \mathbf{V}^{(i)} &= \mathbf{XW}^{(v,i)}\end{aligned}$$

Each head computes its own attention:

$$\mathbf{Y}^{(i)} = \text{Softmax} \left[\frac{\mathbf{Q}^{(i)} \mathbf{K}^{(i)T}}{\sqrt{d_k}} \right] \mathbf{V}^{(i)}$$

The outputs of all heads are concatenated and linearly transformed:

$$\mathbf{Y} = \text{Concat}(\mathbf{Y}^{(1)}, \dots, \mathbf{Y}^{(h)}) \mathbf{W}^{(o)}$$

9.2.2 Why Multi-Head?

- **Different perspectives:** Each head can learn to focus on different types of relationships
- **Parallel processing:** All heads process the input simultaneously

- **Richer representations:** Multiple heads capture more complex patterns
- **Stability:** The $\sqrt{d_k}$ scaling prevents gradients from becoming too small

Example 9.1 (Multi-Head Attention in Action). Consider the sentence: "The bank by the river is beautiful"

- **Head 1** might focus on syntactic relationships (subject-verb)
- **Head 2** might focus on semantic relationships (bank-river)
- **Head 3** might focus on positional relationships
- **Head 4** might focus on morphological patterns

Each head learns different attention patterns, providing a comprehensive understanding of the input.

10 Positional Encoding

10.1 The Need for Position Information

Problem: Permutation Invariance

Transformers are inherently **permutation invariant**—they treat input tokens as an unordered set. However, in sequences like text, **position matters**:

- "The cat sat on the mat" $\neq$ "The mat sat on the cat"
- "I love you" $\neq$ "You love I"

10.2 Solution: Positional Encoding

Definition 10.1 (Positional Encoding). **Positional encoding** adds position-specific information to token embeddings so that the transformer can distinguish between tokens based on their position in the sequence.

10.2.1 Sinusoidal Positional Encoding

The original Transformer paper uses sinusoidal functions:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

where:

- pos is the position in the sequence
- i is the dimension index
- d_{model} is the model dimension

10.2.2 Learned Positional Embeddings

Modern approaches often use learned embeddings:

$$\mathbf{x}_n = \mathbf{x}_n + \mathbf{p}_n$$

where $\mathbf{p}_n$ is a learned vector specific to position n .

10.3 Properties of Good Positional Encoding

Observation. Effective positional encoding should satisfy:

- **Unique for each position:** Different positions should have different encodings
- **Consistent for similar distances:** Positions i and j should have similar encodings if $|i - j|$ is small
- **Generalizable:** Should work for sequences longer than seen during training
- **Additive:** Should not interfere with the semantic meaning of tokens

11 The Complete Transformer Architecture

11.1 Encoder-Decoder Structure

Transformer Architecture

The Transformer consists of an **encoder** and a **decoder**, both composed of stacked layers:

- **Encoder:** Processes the input sequence and creates contextual representations
- **Decoder:** Generates the output sequence using encoder outputs and previously generated tokens

11.1.1 Encoder Stack

Each encoder layer contains:

1. **Multi-head attention:** Self-attention over input tokens
2. **Feed-forward network:** Point-wise transformations
3. **Residual connections and layer normalization**

11.1.2 Decoder Stack

Each decoder layer contains:

1. **Masked multi-head attention:** Self-attention over target tokens (masked to prevent peeking)
2. **Multi-head attention:** Attention over encoder outputs
3. **Feed-forward network**
4. **Residual connections and layer normalization**

11.2 Layer Normalization and Residual Connections

Definition 11.1 (Residual Connection). A **residual connection** adds the input of a layer to its output:

$$\mathbf{y} = \mathbf{x} + \text{Layer}(\mathbf{x})$$

Definition 11.2 (Layer Normalization). **Layer normalization** normalizes the features within each sample:

$$\text{LayerNorm}(\mathbf{x}) = \gamma \cdot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

12 Transformer Layers

12.1 Multi-Head Attention Algorithm

Observation. Multi-head attention allows the model to simultaneously attend to information from different representation subspaces. Each head learns independent query, key, and value transformations, enabling diverse attention patterns.

Algorithm 12.2: Multi-head attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D}$: $\{x_1, \dots, x_N\}$
 Query weight matrices $\{\mathbf{W}_1^{(q)}, \dots, \mathbf{W}_H^{(q)}\} \in \mathbb{R}^{D \times D}$ (one per head)
 Key weight matrices $\{\mathbf{W}_1^{(k)}, \dots, \mathbf{W}_H^{(k)}\} \in \mathbb{R}^{D \times D}$ (one per head)
 Value weight matrices $\{\mathbf{W}_1^{(v)}, \dots, \mathbf{W}_H^{(v)}\} \in \mathbb{R}^{D \times D_v}$ (one per head)
 Output weight matrix $\mathbf{W}^{(o)} \in \mathbb{R}^{HD_v \times D}$ (projects concatenated heads back to original dimension)
Output: $\mathbf{Y} \in \mathbb{R}^{N \times D}$: $\{y_1, \dots, y_N\}$

```
// compute self-attention for each head (Algorithm 12.1)
for  $h = 1, \dots, H$  do
   $\mathbf{Q}_h = \mathbf{XW}_h^{(q)}$ ,  $\mathbf{K}_h = \mathbf{XW}_h^{(k)}$ ,  $\mathbf{V}_h = \mathbf{XW}_h^{(v)}$  // project inputs for this head
   $\mathbf{H}_h = \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h)$  //  $\mathbf{H}_h \in \mathbb{R}^{N \times D_v}$ 
end for
 $\mathbf{H} = \text{Concat}[\mathbf{H}_1, \dots, \mathbf{H}_H]$  // concatenate all head outputs:  $\mathbf{H} \in \mathbb{R}^{N \times HD_v}$ 
return  $\mathbf{Y}(\mathbf{X}) = \mathbf{HW}^{(o)}$  // project back to original dimension
```

12.1.1 Key Insights

- Each head operates independently with its own learnable transformations, allowing different heads to capture different types of relationships
- The concatenation of all heads produces a richer representation than a single attention head
- The output projection $\mathbf{W}^{(o)}$ combines information from all heads and maps back to the original dimension D

12.2 Transformer Layer Structure

Definition 12.1 (Residual Connection). A residual connection (or skip connection) adds the input directly to the output of a layer: $\text{output} = \text{Layer}(\text{input}) + \text{input}$. This enables gradients to flow directly through the network during backpropagation.

- **Multi-head self-attention is the core of transformer layers** - it enables the model to attend to different parts of the input simultaneously
- **To improve training efficiency we introduce residual connections** that bypass the multi-head structure - this helps with gradient flow and allows information to propagate

directly through layers

- **Then, layer normalization** (Ba, Kiros, and Hinton, 2016) is performed, which also improves training efficiency - it stabilizes training by normalizing the distribution of layer inputs
- **Resulting transformation:**

$$\mathbf{Z} = \text{LayerNorm}[\mathbf{Y}(\mathbf{X}) + \mathbf{X}]$$

This combines the attention output with the original input and normalizes the result.

12.3 Nonlinearity and Feed-Forward Network

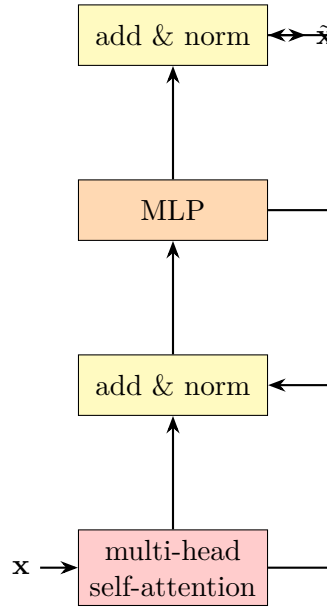
Observation. The attention mechanism, while powerful, has a fundamental limitation: it produces outputs that are linear combinations of the input value vectors. This constrains the expressiveness of the layer.

- **Problem:** the attention mechanism creates linear combinations of the value vectors, which are linearly combined to produce output vectors. So, **outputs of attention layers are constrained to be linear combinations of the inputs.**
- **Nonlinearity enters only in the attention weights (softmax function) but the output vectors are constrained to lie in the subspace spanned by the input vectors, which limits the expressiveness of the attention layer**
- **Solution:** We can enhance the flexibility by **post-processing the output of each layer using a standard nonlinear neural network with D inputs and D outputs** ($\text{MLP}[\cdot]$). E.g., two-layer fully connected with ReLU hidden units
 - This feed-forward network introduces nonlinearity that allows the model to learn more complex transformations
 - The MLP typically has a hidden layer with more units than D (e.g., $4D$) to increase model capacity
- **Notice:** the same shared network is applied to each of the output vectors (i.e., tokens, rows of $\mathbf{Z}$) - this is called a position-wise feed-forward network
- **Residual connections and layer normalization** are applied also to this stage, to improve training efficiency and enable deeper networks

12.4 Complete Transformer Layer

Final output of the transformer layer

$$\tilde{\mathbf{X}} = \text{LayerNorm}[\text{MLP}[\mathbf{Z}] + \mathbf{Z}]$$



Courtesy of Bishop et al., 2024

12.5 Transformer Layer Algorithm

Observation. A complete transformer layer combines multi-head attention with a feed-forward network, connected via residual connections and layer normalization. This design enables the model to learn both attention-based relationships and complex nonlinear transformations.

Algorithm 12.3: Transformer layer

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D}$: $\{x_1, \dots, x_N\}$

Multi-head self-attention layer parameters (from Algorithm 12.2)

Feed-forward network parameters (typically a 2-layer MLP with ReLU activation)

Output: $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times D}$: $\{\tilde{x}_1, \dots, \tilde{x}_N\}$ (same dimension as input)

$\mathbf{Z} = \text{LayerNorm}[\mathbf{Y}(\mathbf{X}) + \mathbf{X}]$ // Apply attention, add residual, normalize

$\tilde{\mathbf{X}} = \text{LayerNorm}[\text{MLP}[\mathbf{Z}] + \mathbf{Z}]$ // Apply MLP, add residual, normalize

return $\tilde{\mathbf{X}}$

12.5.1 Design Rationale

- **Residual connections:** Enable gradients to flow directly through the network, facilitating training of deep models
- **Layer normalization:** Stabilizes training by normalizing activations, reducing internal covariate shift
- **Post-norm architecture:** Normalizing after each sub-layer (attention and MLP) has been shown to improve training stability
- **Dimension preservation:** The output has the same dimension as the input, allowing layers to be stacked sequentially

13 Computational Complexity of the Attention Layer

13.1 Attention Layer Complexity

Notation

N vectors (tokens), D dimension

- **Attention layer**
 - Matrices $\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, and $\mathbf{W}^{(v)}$ have $\mathcal{O}(D^2)$ parameters and are shared across input tokens
 - Matrices $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ have $\mathcal{O}(ND)$ parameters
 - **Total computational cost for attention** ($\text{Softmax}[\frac{\mathbf{QK}^T}{\sqrt{D_k}}] \mathbf{V}$): $\mathcal{O}(N^2D)$
- **MLP (after attention)**
 - D inputs, D outputs
 - Computational cost of a forward pass (for 1 token): $\mathcal{O}(D^2)$
 - **Total computational cost (for N tokens)**: $\mathcal{O}(ND^2)$
- Depending on N and D the transformer layer or the MLP layer may dominate the computational cost

13.2 Comparison with Fully Connected Network

Notation

N vectors (tokens), D dimension

- **Standard fully connected network**
 - Input/output size: ND
 - Number of independent parameters: $\mathcal{O}(N^2D^2)$
 - Computational cost of a forward pass: $\mathcal{O}(N^2D^2)$
- **Conclusion:** Compared to a standard fully connected network (complexity $\mathcal{O}(N^2D^2)$) a transformer layer (complexity $\max(\mathcal{O}(ND^2), \mathcal{O}(N^2D))$) is computationally more efficient

14 Tokenization: One-Hot Representation

14.1 How to Compute Token Vectors for a Given Text?

- Example: *"Hello, I'm a language model,"*

14.2 One-Hot Representation

- Fixed dictionary of words
- Vectors of length equal to the dictionary size
- k -th word has 1 in position k , 0 elsewhere
- **Problem:** Huge dimensionality of the dictionary and the tokens

15 Tokenization: Byte Pair Encoding

15.1 Motivation

One-Hot Encoding Issues

The **one-hot encoding** approach has a large dictionary size, which can become inefficient. To address this issue, we could use a dictionary of characters instead of words. However, this approach would discard the **semantically important word structure** of the language.

15.2 Byte Pair Encoding (BPE)

Definition: Byte Pair Encoding

Byte Pair Encoding (BPE) provides an intermediate solution, acting as a compromise between character-level and word-level tokenization. BPE is a compression method adapted for tokenization. The process works as follows:

1. Start with individual characters.
2. Search a body of text for the most frequently occurring adjacent pairs of tokens.
3. Replace the adjacent pair with a new token and add the token to the dictionary (note: new tokens cannot contain spaces).
4. Repeat the process until a predefined number of tokens is reached.

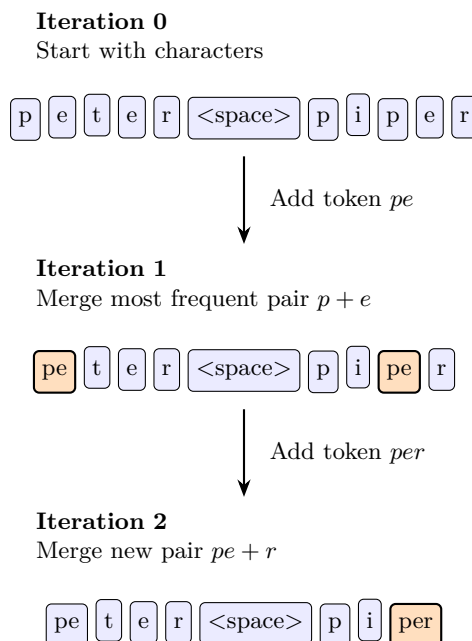


Figure 1: Byte Pair Encoding iteratively merges the most frequent adjacent tokens, growing the vocabulary step by step.

15.3 Dictionary Generation with Byte Pair Encoding

Consider the text: “Peter Piper picked a peck of pickled peppers.”

After applying Byte Pair Encoding, frequently occurring pairs of characters (e.g., “pe” and “ck”) will be merged into tokens and added to the dictionary.

This method enables more efficient tokenization while retaining semantic structure.

15.4 Text Tokenization

An example of tokenization using Byte Pair Encoding is the sentence: *“Hello, I’m a language model.”*

Tokens: [15496, 11, 314, 479, 257, 3306, 3560, 11]

Where:

- **Token ID 15496:** "Hello" (word without leading space)
- **Token ID 11:** ", " (comma)
- **Token ID 314:** "GI" (leading-space marker → "I")
- **Token ID 479:** "'m" (apostrophe + m)
- **Token ID 257:** "a"
- **Token ID 3306:** "language"
- **Token ID 3560:** "model"
- **Token ID 11:** ", " (final comma)

Token ID	BPE Piece	Meaning / Note
15496	Hello	Word without leading space
11	,	Comma
314	GI	Leading-space marker → “I”
479	'm	Apostrophe + m
257	a	“a”
3306	language	“language”
3560	model	“model”
11	,	Final comma

Table 1: Byte Pair Encoding Tokenization Example

16 Transformer Architectures

16.1 Introduction to Transformer Architectures

The flexibility of transformer processing layers has allowed the development of Large Language Models (LLMs). There are three main transformer-based architectures:

- **Encoder:** Input: sequence of tokens (e.g., text); Output: sequence of tokens (embedding) usable for a downstream task (e.g., sentiment classification).
- **Decoder:** Input: single vector (e.g., text prompt or image); Output: sequence of tokens (e.g., next text or image caption).
- **Encoder+Decoder:** Input: sequence (e.g., sequence in Italian); Output: sequence (e.g., sequence in English).

These architectures are the foundation of many powerful transformer-based models like GPT, BERT, and other state-of-the-art models in natural language processing.

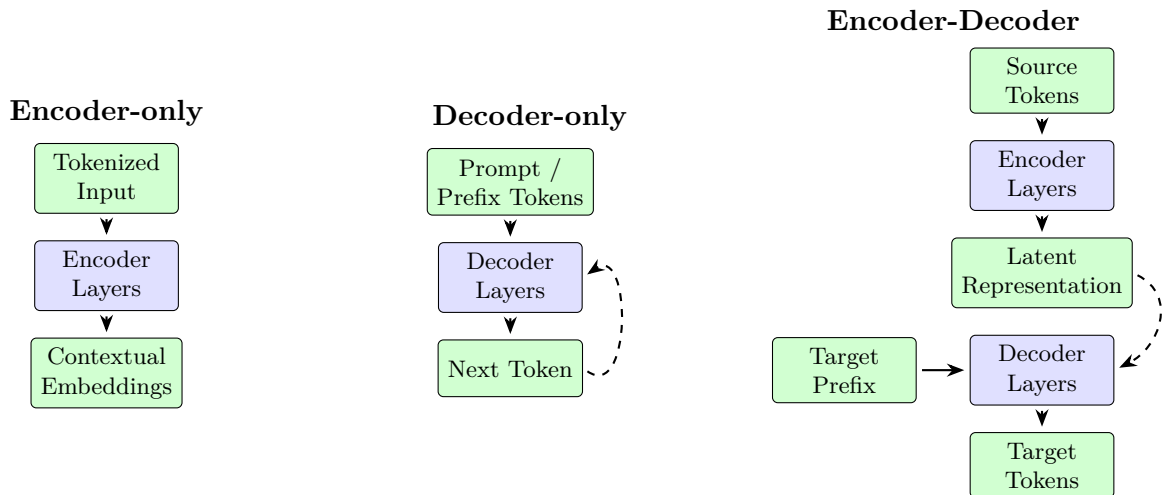


Figure 2: Key transformer configurations: encoder models for understanding, decoder models for autoregressive generation, and encoder-decoder models for sequence transduction.

16.2 Decoder Transformers

Decoder Transformer: Key Idea

- **Generative model:** produces an output sequence token by token, as in GPT-style language models.
- **Autoregressive factorization:** models the joint distribution over tokens by multiplying conditional probabilities that depend only on previous tokens.
- **Feedback loop:** each sampled token is appended to the prefix and fed back to predict the next token, enabling open-ended generation.

16.2.1 Autoregressive Objective

A decoder transformer constructs a probabilistic model over a sequence $\mathbf{x}_{1:N} = (x_1, \dots, x_N)$:

$$p(\mathbf{x}_{1:N}) = \prod_{n=1}^N p(x_n \mid x_1, \dots, x_{n-1})$$

Each conditional distribution $p(x_n \mid x_1, \dots, x_{n-1})$ is parameterized by the transformer and learned from data.

- **Input to the network:** the prefix of $n - 1$ tokens.
- **Output:** a categorical distribution over the vocabulary for token n .
- **Sampling:** drawing from this distribution yields the next token, which is appended to the sequence to predict x_{n+1} , and so on.

16.2.2 Architecture Overview

The decoder is composed of a stack of L identical transformer layers followed by a projection head:

- **Token embeddings:** each input token (including a start-of-sequence symbol) is mapped to a D -dimensional vector, and a positional encoding is added to retain order information.
- **Masked self-attention layers:** each layer mixes information across tokens while masking future positions, ensuring that predictions depend only on past tokens.

- **Feed-forward sublayers:** position-wise networks refine each token representation after attention. Residual connections and layer normalization (not shown in the slides) stabilize deep stacks.

Linear Softmax Module (LSM)

Let $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times D}$ be the output of the last transformer layer. A learned matrix $W^{(p)} \in \mathbb{R}^{D \times K}$ maps each token representation to the vocabulary of size K :

$$\mathbf{Y} = \text{Softmax}(\tilde{\mathbf{X}}W^{(p)})$$

The softmax is applied row-wise, yielding a probability distribution over tokens for every position. Each position's prediction is trained with a cross-entropy loss comparing the predicted distribution with the true token.

16.2.3 Training Decoder Transformers

- **Self-supervised objective:** large corpora of raw text provide sequences where the model learns to predict the next token without manual labels.
- **Training sample:** an input sequence $(x_1, \dots, x_n)$ is paired with the target x_{n+1} (often shifted versions of the same sequence).
- **Loss function:** cross-entropy values are summed across tokens and mini-batches, encouraging high probability for the correct next token at every position.
- **Batching strategy:** samples are treated as independent, yet efficiency improves by processing complete sequences in parallel on modern accelerators.

Constructing Training Pairs Consider the sentence “I swam across the river to get to the other bank.”. Two example training pairs are:

(“I swam across”, “the”)
 (“I swam across the”, “river”)

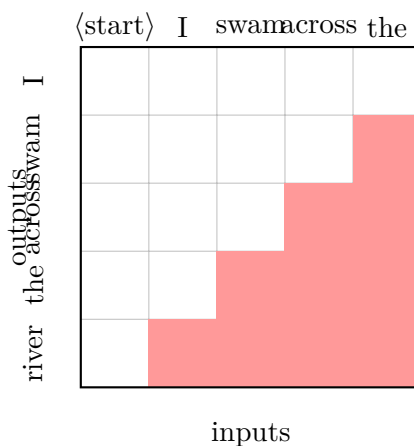
To prevent the model from copying the next token directly when trained in parallel, two precautions are required.

Parallel Training Precautions

1. **Input shift:** the sequence fed to the network is shifted by one position (input token x_n predicts output y_{n+1}), and a special $\langle \text{start} \rangle$ token is prepended.
2. **Masked attention:** the attention mechanism is restricted so that a position can only attend to itself and earlier tokens, blocking any “peek ahead” behavior.

16.2.4 Masked Attention

- All attention weights that would connect a token to *future* positions are forced to zero before the softmax normalization.
- In matrix form this corresponds to zeroing the entries above the main diagonal in the attention score matrix and then renormalizing each row so that it still sums to one.
- The triangular mask ensures that, for every position, the prediction is conditioned only on the prefix observed so far.



16.2.5 Batching and Padding

- For efficient GPU utilization, multiple sequences are combined into an input tensor and processed in a single batch.
- Batching requires each sequence to have the same length, so shorter sequences are padded with a special $\langle \text{pad} \rangle$ token.
- A **padding mask** is added to the attention computation so that tokens representing real content never attend to padded positions.

16.2.6 Using the Trained Decoder

- The model outputs a probability distribution over the vocabulary via the softmax layer, representing the likelihood of each possible next token given the current prefix.
- After sampling or selecting the next token, the extended sequence is fed back through the decoder to generate the subsequent token; this loop continues until an end-of-sequence token is produced.
- Thanks to masked attention, the representation of an existing token depends only on earlier tokens and therefore does not need to be recomputed when later tokens are generated, enabling reuse of intermediate activations.